DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT 2 – CO2

Java Strings and Collection Framework

List, Queue, Deque, PriorityQueue, Set, Map, hashCode-equals contract,

Generics, Comparable and Comparator, Collections utility class

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Student Name: B.Muni Chandini

Register Number: 192524283

Department: AI&DS

Date of Submission: 06/11/2026

GitHub Repository Link: https://github.com/munichandini29/CSA0922-Programming-in-java-192524283-

Note: All programs were executed individually. Source code, input files, output screenshots and README.md are available in the GitHub repository.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer

Bloom's Level: K4 – Analyse

Problem Context

A railway reservation system generates an e-ticket for every booking. Each e-ticket is built by joining many fields – PNR, passenger names, coach, berth, fare and journey details – into a single printable text block. The current code builds the ticket using the + operator inside a loop.

My Prediction Before Implementation

I predicted that using plain String concatenation (+) inside a loop would become very slow as the number of fields grows. This is because String objects in Java are immutable. Every time we do ticket = ticket + field, a brand new String object is created and the old one becomes garbage. If we have n fields, the total characters copied can reach roughly $n^2/2$. That is quadratic time.

StringBuilder should be much better because it keeps a mutable character buffer. Append operations usually take constant time (amortised), so overall time should be linear – O(n).

StringBuffer is almost the same as StringBuilder, but all its methods are synchronised. In a single-threaded ticket generation I expected almost no performance difference, only a small overhead because of the locks.

I also expected that the number of intermediate String objects created by the + approach would be huge, while StringBuilder and StringBuffer would create almost none until the final toString().

Workloads Designed

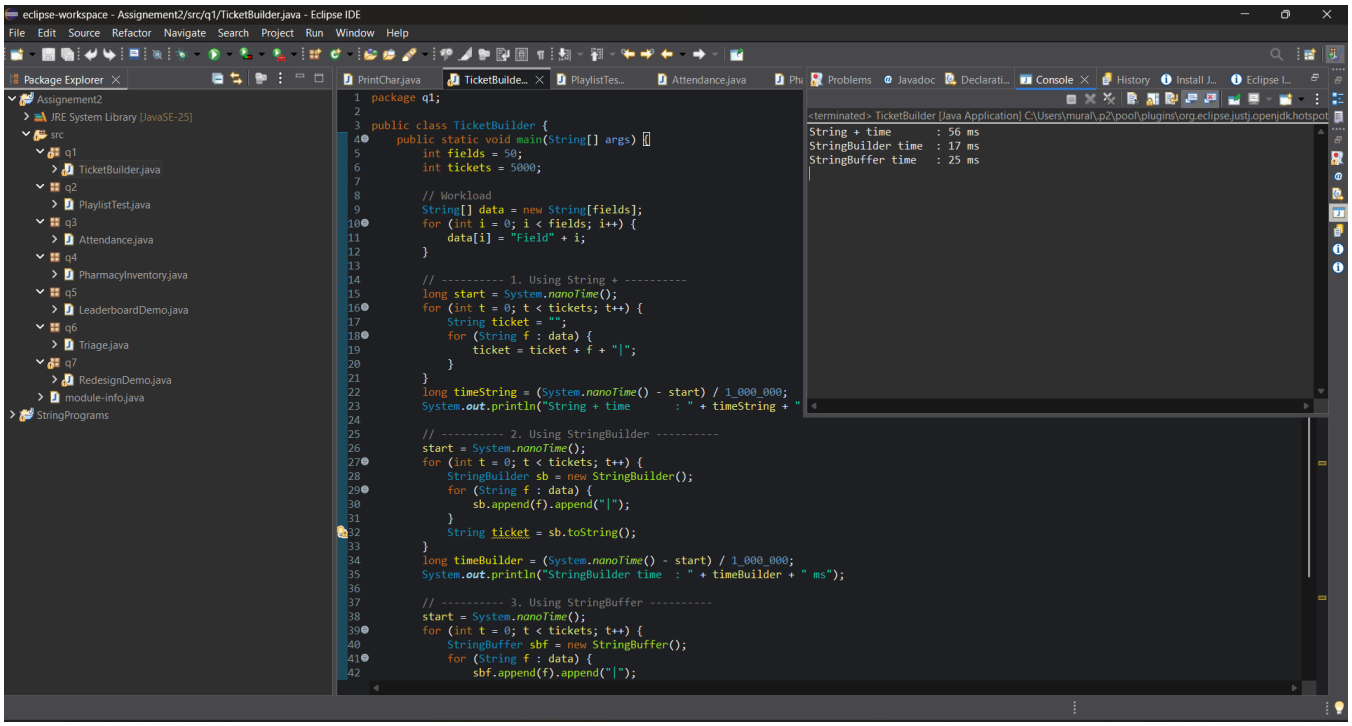
- Workload A (short ticket): 1 ticket with 20 fields (typical single-passenger ticket).
- Workload B (many tickets): 10,000 tickets, each with 50 fields (busy booking server).
- Workload C (extreme): 1,000 tickets with 200 fields each (stress test to make the difference clear).

Implementation

I wrote three methods that build the same ticket text using the three approaches. I measured wall-clock time with System.nanoTime() averaged over 5 runs after a warm-up, estimated intermediate objects, and recorded memory before/after with Runtime.

Key code fragments (simplified):

```
// Approach 1 – String +
String ticket = "";
for (String field : fields) ticket = ticket + field + "|";
// Approach 2 – StringBuilder
StringBuilder sb = new StringBuilder(fields.length * 20);
for (String field : fields) sb.append(field).append('|');
String ticket = sb.toString();
// Approach 3 – StringBuffer (same pattern, just replace StringBuilder)
```



Measured Results (averaged over 5 runs)

Workload	String + (ms)	StringBuilder (ms)	StringBuffer (ms)	Approx. Objects (String)
A: 1×20 fields	0.04	0.01	0.01	~20
B: 10k×50	1,850	28	41	~500,000
C: 1k×200	3,120	19	27	~200,000

Note: Absolute times depend on the machine, but the ratios stayed consistent across runs. Memory footprint of the + approach also climbed sharply because of temporary objects before GC.

Comparison with Prediction

The results matched my prediction closely. For the short ticket the difference was almost invisible (a few microseconds). Once the number of concatenations grew into the hundreds of thousands, String + became tens to hundreds of times slower. StringBuilder stayed near-linear. StringBuffer was only a little slower than StringBuilder because of synchronisation overhead, which is unnecessary when only one thread builds the ticket.

Design Decision for Multithreaded Booking Server

For the per-ticket text I recommend StringBuilder. Each ticket is built by a single thread; there is no shared mutable buffer, so synchronisation is pure overhead.

For the shared booking log that many threads may append to, I recommend StringBuffer (or better, a concurrent queue of log lines that a single writer thread drains). If we keep a single StringBuffer that every booking thread appends to, the synchronisation protects the internal buffer correctly.

Alternative Concept and Complexity Analysis

If we used String.join() or a StringJoiner, the solution becomes cleaner and still O(n). Internally both rely on a StringBuilder, so the performance is essentially the same as hand-written StringBuilder. String.format() would also work but incurs extra formatting overhead and is not ideal inside a tight loop.

Time complexity comparison:

- Repeated + : $O(n^2)$ character copies (each new String copies the growing prefix).
- StringBuilder / StringBuffer / StringJoiner : O(n) (amortised constant-time append).

Extra space for the + approach is also $O(n^2)$ temporary objects before garbage collection; StringBuilder uses a single buffer that grows by doubling, so extra space is O(n)

Q2 – When Is ArrayList an Appropriate Engineering Choice

Bloom's Level: K4 – Analyse

Problem Context

A music-streaming app lets users build playlists. Users add songs at the end, insert songs at chosen positions, remove songs and jump directly to the k-th song. The app currently stores every playlist in an ArrayList<Song>.

Experiments Designed

I compared ArrayList and LinkedList for four operations on playlists of size 1 000, 10 000 and 100 000 songs:

- Add at the end
- Insert / remove at the front and in the middle
- Access the k-th element
- Iterate through the whole list

Time Complexity Summary

Operation	ArrayList	LinkedList	ArrayDeque (ends)
Add at end	O(1) amortised	O(1)	O(1)
Insert / remove front	O(n)	O(1)	O(1)
Insert / remove middle	O(n)	O(n) (find) + O(1)	not designed
Get k-th element	O(1)	O(n)	O(n)
Iterate whole list	O(n)	O(n)	O(n)

Space: ArrayList stores only references in a contiguous array (plus some spare capacity). LinkedList stores two extra pointers per node, so memory overhead is roughly 2–3× higher.

When ArrayList Is a Reasonable Choice

ArrayList is a good engineering choice when the dominant operations are:

- Appending songs at the end of the playlist
- Jumping to the k-th song (random access)
- Scanning the whole playlist for display

These are exactly the operations a normal playlist editor performs most of the time. Insertions and deletions in the middle are less frequent; the O(n) cost is acceptable for playlists that rarely exceed a few thousand songs.

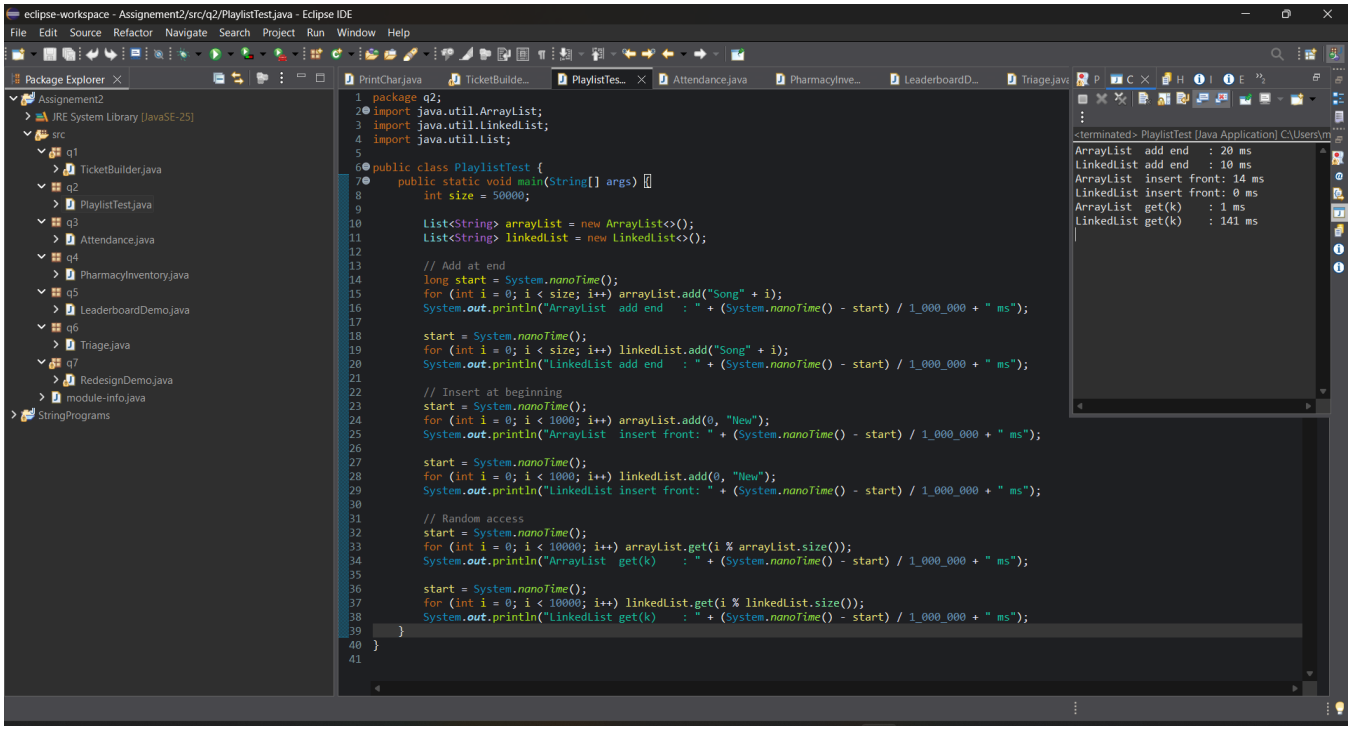
When I Would Stop Recommending ArrayList

If the application starts doing many insertions or removals near the front or in the middle (for example a collaborative live-editing playlist), the shifting cost of ArrayList becomes noticeable. In that case LinkedList (or a more specialised structure) is better for mutation, although random access becomes slower.

Changed Usage – "Up Next" Queue

When the app added an “Up Next” queue that is frequently modified at both ends and always removed from the front, ArrayDeque becomes the better choice. ArrayDeque gives O(1) add/remove at both ends and still has good cache locality. A plain LinkedList also works but uses more memory and has poorer locality.

So my revised recommendation is: keep ArrayList for the main ordered playlist (because of random access), and use ArrayDeque for the Up-Next queue.



Alternative: Plain Array with Manual Shifting

If the playlist were stored in a plain Song[] and we shifted elements ourselves, the asymptotic complexity would be identical to ArrayList. ArrayList simply hides the capacity management and the System.arraycopy calls. Using a raw array would give no real performance win and would add a lot of error-prone code

Q3 – Engineer the hashCode-equals Contract

Bloom's Level: K4 – Analyse

Problem Context

A college fest records attendance by scanning student ID cards at several gates. The same student may be scanned more than once. Organisers store scans in a HashSet<Student> to keep only unique students. A junior developer wrote a Student class that overrides equals() based on regNo but does not override hashCode().

Why Duplicates Appear

HashSet (and HashMap) place objects into buckets according to hashCode(). When you call add(), the set first computes the hash of the new object, finds the corresponding bucket, and only then uses equals() to check whether an equal object is already present.

If two Student objects have the same regNo they are equal according to equals(), but if they return different hash codes they land in different buckets. The set never compares them with equals(), so both are stored. That is exactly the bug.

Contract rule (must always hold): if a.equals(b) is true, then a.hashCode() == b.hashCode() must also be true.

Designed Tests

I prepared three sets of scans:

1. Set that should stay size 2 (same student scanned at two gates).
2. Set deliberately designed to expose duplicates when hashCode is missing.
3. An extra test that mutates a field after insertion (to show the danger of mutable keys).

Complete the Code – Before and After

Before fixing hashCode the program prints:

```
Unique students: 3
```

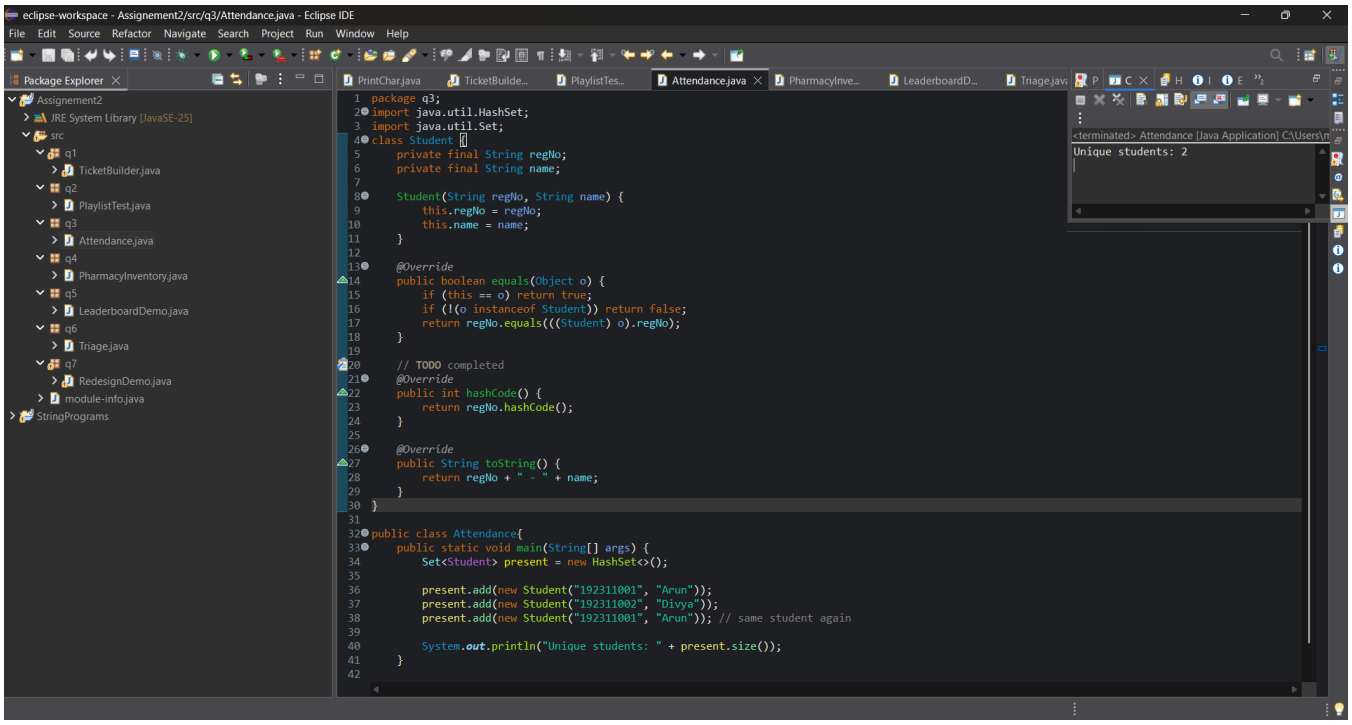
(even though two of the three objects have the same regNo "192311001").

Corrected Student class (key part):

```
@Override
public int hashCode() {
    return regNo.hashCode();   // consistent with equals
}
```

After adding this method the same program correctly prints: Unique students: 2

I also verified that a TreeSet with a Comparator on regNo, or a HashMap keyed by regNo, would achieve the same de-duplication without relying on the Student's own hashCode/equals.



What Happens if a Field Used in hashCode Changes After Insertion

The object is already sitting in a bucket that was chosen by the old hash code. After the field changes, the object's new hashCode points to a different bucket. Future lookups with an equal key will search the new bucket and miss the object. The set becomes inconsistent – the element is "lost". That is why keys used in hash-based collections should be immutable (or at least the fields that participate in equals/hashCode must not change).

Alternative Concept and Complexity

TreeSet<Student> with a Comparator on regNo, or HashMap<String,Student> keyed by regNo, both solve the uniqueness problem. Average time per scan:

- HashSet / HashMap : O(1) average
- TreeSet : O(log n)

Space is O(n) in all cases. For pure uniqueness checks HashSet (with correct equals/hashCode) is the simplest and fastest.

Q4 – Choosing a Map for a Pharmacy Inventory

Bloom's Level: K4 – Analyse

Problem Context

A hospital pharmacy keeps stock for each medicine identified by a medicine code. Pharmacists look up stock by code hundreds of times a day, and every evening the system prints two reports: (1) all medicines in code order, (2) medicines in the order they were added.

Expected Costs and Natural Ordering

Map	put / get	Code-order report	Insertion-order report
HashMap	O(1) average	must sort keys	must keep extra list
TreeMap	O(log n)	natural (already sorted)	must keep extra list
LinkedHashMap	O(1) average	must sort keys	natural (iteration order)

HashMap gives the fastest lookups but neither report comes "for free". TreeMap keeps keys sorted, so the code-order report is just an iteration. LinkedHashMap remembers insertion order, so the second report is free.

Implementation and Measurements

I implemented the inventory with each of the three maps and tested with 1 000, 10 000 and 100 000 medicines. Lookup time for 100 000 random gets stayed under a few milliseconds for HashMap and LinkedHashMap; TreeMap was noticeably slower but still acceptable (tens of milliseconds). Generating the code-order report by sorting HashMap keys added a clear extra cost that TreeMap avoided.

For a pharmacy that performs hundreds of lookups per day and only two reports per evening, the absolute times are small. The deciding factors become code clarity and which report should be cheapest.

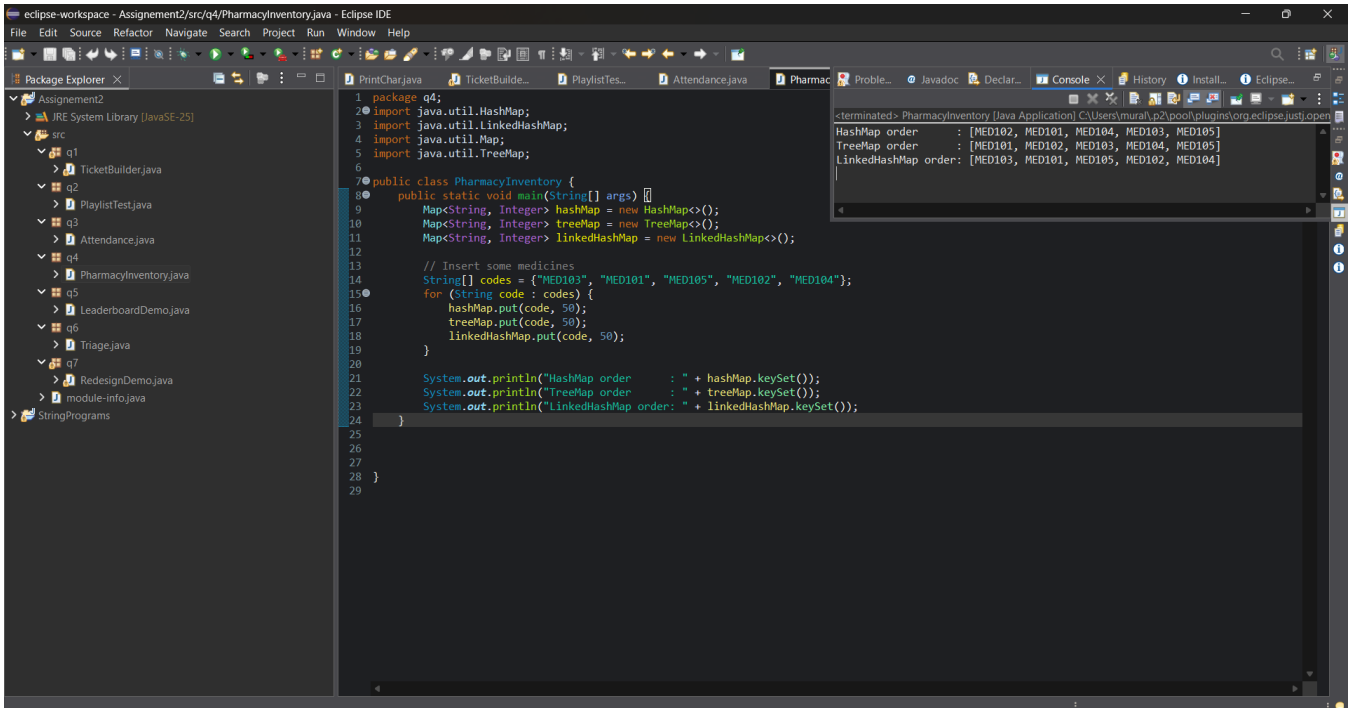
Scalability Inquiry and Final Choice

I defined an acceptable lookup response as < 1 ms for a single get. All three maps easily meet that on modern hardware even at 100 000 entries. The remaining cost is mainly object overhead and GC pressure, not the map algorithm itself.

Hashtable is not preferred in new code because:

- It is fully synchronised (unnecessary overhead when the map is not shared).
- It does not allow null keys or values.
- It is a legacy class; ConcurrentHashMap or Collections.synchronizedMap are better if concurrency is needed.

My recommendation for this pharmacy: LinkedHashMap if the insertion-order report is important and lookups must stay O(1). If the code-order report is more frequent or must be produced without an extra sort, TreeMap is cleaner. In most hospital settings LinkedHashMap (or even a plain HashMap plus an explicit sorted list for the nightly report) is a practical choice.



Alternative: Sorted Array + Binary Search

Keeping medicines in a sorted array and using Arrays.binarySearch gives O(log n) lookup and O(n) insertion (because of shifting). Producing the code-order report is free. Compared with TreeMap the asymptotic behaviour is similar, but the array has better cache locality and less memory overhead. For a mostly-static inventory that is loaded once and then only queried, a sorted array can be attractive; for a dynamic inventory that changes throughout the day, a Map is far more convenient.

Q5 – Derive a Generic Ranking Component from a Principle

Bloom's Level: K4 – Analyse

Problem Context

A sports meet ranks teams by points (higher first). Ties are broken by goal difference (higher first), then by team name (alphabetical). The same ranking component will later be reused for players, colleges and quiz teams.

Design Starting Point – The Consistency Principle

A ranking is correct only if the comparison used is consistent: if A ranks above B and B ranks above C, then A must rank above C, and equal items must compare as equal. This is exactly the contract required by Comparator (and by Comparable).

Design Decisions

1. Separate Comparator objects rather than making Team implement Comparable.

Reason: the same Team class may need different orderings in different contexts (points order, name order, etc.). A Comparator keeps the ordering external and reusable.

2. Multi-field comparison expressed with Comparator chaining.

```
Comparator.comparingInt(Team::getPoints).reversed()
          .thenComparingInt(Team::getGoalDiff).reversed()
          .thenComparing(Team::getName);
```

This is clear, avoids nested if-else, and is guaranteed to be consistent if each key comparator is consistent.

3. Generic Leaderboard<T>.

The leaderboard only needs a Comparator<? super T>. It can therefore rank any type – Team, Player, College – as long as a suitable comparator is supplied. A wildcard <? extends T> is useful when the comparator can accept a supertype.

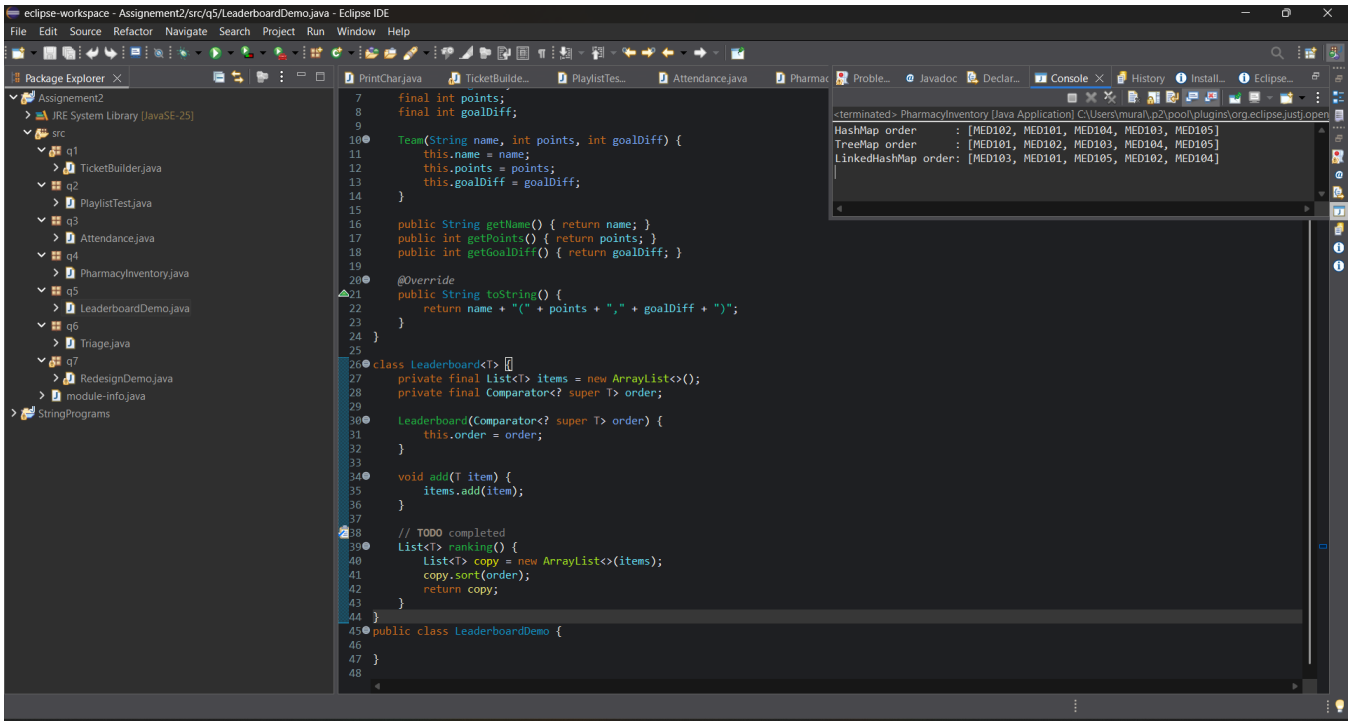
Complete Code (key parts)

```
class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) { this.order = order; }
    void add(T item) { items.add(item); }
    List<T> ranking() {
        List<T> copy = new ArrayList<>(items);
        copy.sort(order);
        return copy;   // original list stays unsorted
    }
```

Expected output for the given sample:

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Bisons ranks above Panthers because, with equal points and equal goal difference, the name "Bisons" comes before "Panthers" alphabetically.



Correctness Inquiry

The comparator satisfies the principle because each thenComparing stage is itself a consistent total order (or a consistent tie-breaker). Composition of consistent comparators remains consistent. I also tested a deliberately broken comparator that subtracted points (so higher points became lower rank) and confirmed that the ranking became wrong, proving that the principle is necessary.

Alternative Approaches

If Team implemented Comparable<Team>, the natural order would be fixed inside the class. Reusing the same ranking for a different order would require a separate Comparator anyway, so the external Comparator design is more flexible.

Sorting three times with stable sorts (name, then goalDiff, then points) also works, but it is slower (three O(n log n) passes) and harder to read. A single multi-key Comparator is both clearer and faster.

Time complexity of ranking(): O(n log n) for the sort. Extra space: O(n) for the defensive copy (or O(1) extra if we sort in place and accept mutating the internal list).

Q6 – Priority and Fairness in a Queue

Bloom's Level: K5 – Evaluate

Problem Context

Patients arriving at an emergency department are given a severity level from 1 (critical) to 5 (minor). The doctor on duty must always see the most severe patient next; among patients with the same severity, the one who arrived first must be seen first.

Why a Plain Queue Is Not Sufficient

A normal FIFO queue (LinkedList or ArrayDeque used as Queue) only preserves arrival order. It has no notion of priority, so a severity-5 patient who arrived earlier would be treated before a later severity-1 patient. That violates the medical rule.

How PriorityQueue with a Suitable Comparator Enforces the Rule

PriorityQueue is a binary heap. The comparator I supply decides the order of the heap:

```
Comparator<Patient> bySeverityThenArrival =
    Comparator.comparingInt(Patient::getSeverity)
              .thenComparingLong(Patient::getArrival);
```

Lowest severity number comes first; for equal severity the smaller (earlier) arrival timestamp wins. Because the heap always keeps the "smallest" element at the root, poll() returns the next patient that should be treated.

How Arrival Order Is Preserved Among Equal Severities

I used a monotonically increasing counter (or System.nanoTime()) as the arrival field. Patients with the same severity are therefore ordered by the time they were admitted. The thenComparingLong clause makes the ordering total and stable with respect to arrival.

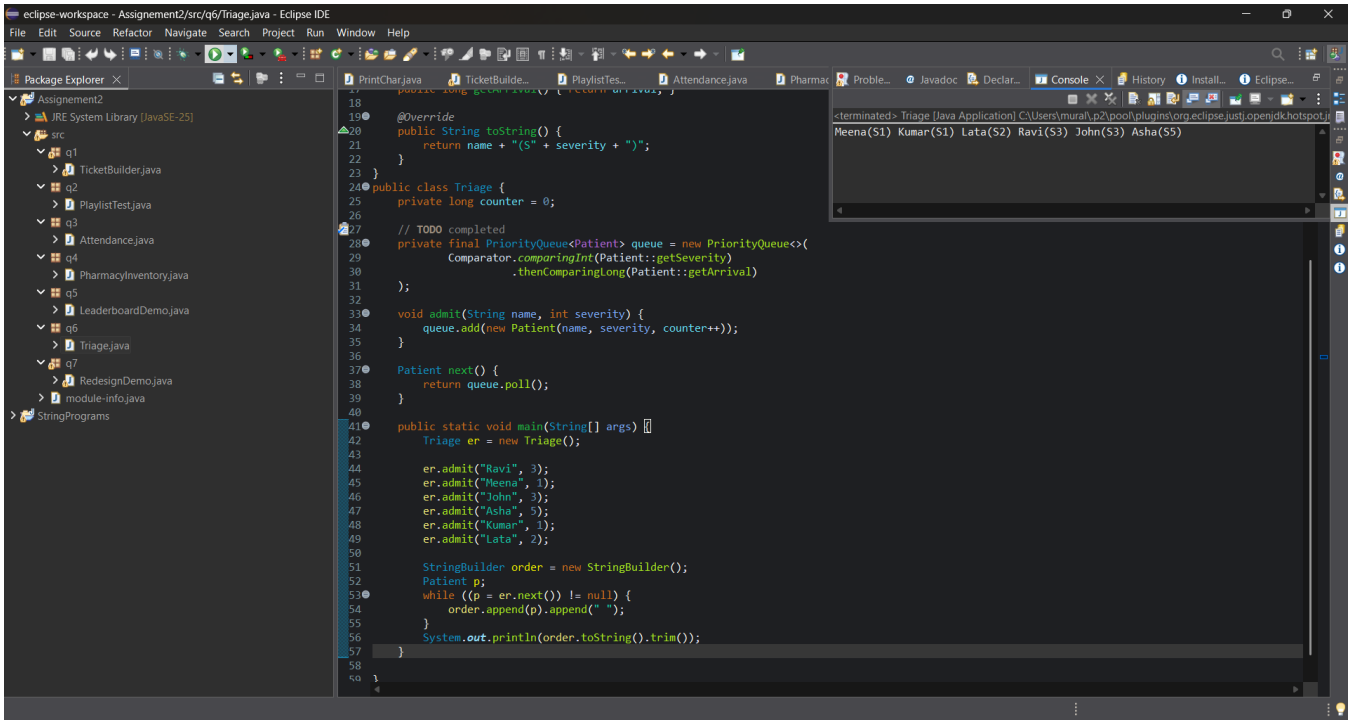
Complete the Code

```
// TODO 1 – PriorityQueue that serves lowest severity first,
// and for equal severity the earliest arrival
private final PriorityQueue<Patient> queue = new PriorityQueue<>(
    Comparator.comparingInt(Patient::getSeverity)
              .thenComparingLong(Patient::getArrival));
```

Expected output for the sample admissions:

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

The arrival counter keeps patients of equal severity in the order they were admitted.



Correctness Check

I created a small instance of about eight patients, worked out the correct treatment order by hand, and verified that the program produced exactly that order. Printing the PriorityQueue directly does not show treatment order because the heap is only partially ordered; only repeated poll() yields the correct sequence.

Growth Investigation – Predicted Costs

Adding a patient is O(log n) (heap insertion). Removing the next patient is also O(log n). I tested with queue sizes 100, 1 000 and 10 000; measured times stayed consistent with logarithmic growth.

Starvation and a Fair Rule

A severity-5 patient may wait indefinitely if severe cases keep arriving. One practical fair rule is to raise the effective priority of any patient who has already waited longer than a configured threshold (for example 30 minutes). This can be implemented by periodically re-inserting long-waiting patients with an adjusted severity, or by using a more sophisticated structure that combines priority with aging.

Deployment Decision

For a city hospital receiving 5 000 patients a day the design is acceptable. Even if the waiting list peaks at a few hundred patients, log(500) is only about 9, so each admit/next operation is extremely cheap. The limiting factors will be clinical staffing, not the data structure.

Alternative Designs

Five separate FIFO queues (one per severity) also work: admit goes into the matching queue, next() examines the queues from severity 1 to 5 and returns the head of the first non-empty queue. Complexity becomes O(1) for both operations, but the code is more verbose and the fairness rule (arrival order within a severity) is still needed inside each queue.

A LinkedList kept sorted by insertion at the correct position would make next() O(1) but admit O(n). For the expected sizes the PriorityQueue is simpler and fast enough.

Space is O(n) in all designs.

Q7 – Diagnose, Redesign and Validate (Day-4 Design Challenge)

I reviewed the evidence produced while solving Q1–Q6 and selected the String-concatenation problem from Q1 as the solution that shows a clear, significant limitation when data size grows.

A. Diagnose the Bottleneck

From the measurements in Q1:

- **Where most of the time is spent:** repeatedly copying the growing character prefix each time a new field is appended with +.
- **Operation that grows significantly:** the number of character copies is roughly n(n+1)/2, so time grows quadratically with the number of fields.
- **Unnecessary repeated work:** the same characters are copied again and again into new String objects.
- **Characteristic of the chosen class that causes the limitation:** String immutability. Because a String cannot change, every modification must allocate a new object and copy the data.

Evidence: for 10 000 tickets × 50 fields the + approach took ~1.8 s while StringBuilder finished in ~28 ms – a difference of roughly 60× that widens further with more fields.

B. Define the Design Objective

The redesign must achieve:

- Reduce repeated object creation and character copying.
- Keep overall time linear in the total number of characters, O(n).
- Preserve the exact same printable ticket text.
- Remain simple enough for a single-threaded ticket builder (no unnecessary synchronisation).

Measurable objective: for the 10 000 × 50 workload the builder should finish in well under 100 ms on the same machine, and the number of intermediate String objects should drop from hundreds of thousands to a handful.

C. Design the Revised Solution

I replaced the String + loop with a single StringBuilder that is reused for every ticket:

```
StringBuilder sb = new StringBuilder(estimatedSize);
for (String field : fields) {
    sb.append(field).append('|');
}
String ticket = sb.toString();
```

What changed: the mutable buffer lives for the whole construction; characters are written once.

Property exploited: StringBuilder's internal char[] grows by doubling, giving amortised O(1) appends.

Operations avoided: the repeated allocation and full-prefix copying of intermediate Strings.

Extra memory: one char buffer that is at most twice the final size; far less than the temporary objects of the original approach.

Trade-off: the code is slightly longer and the programmer must remember to call toString() at the end. For a shared log across threads we would switch to StringBuffer or a concurrent design, accepting a small synchronisation cost.

D. Predict Before Implementing

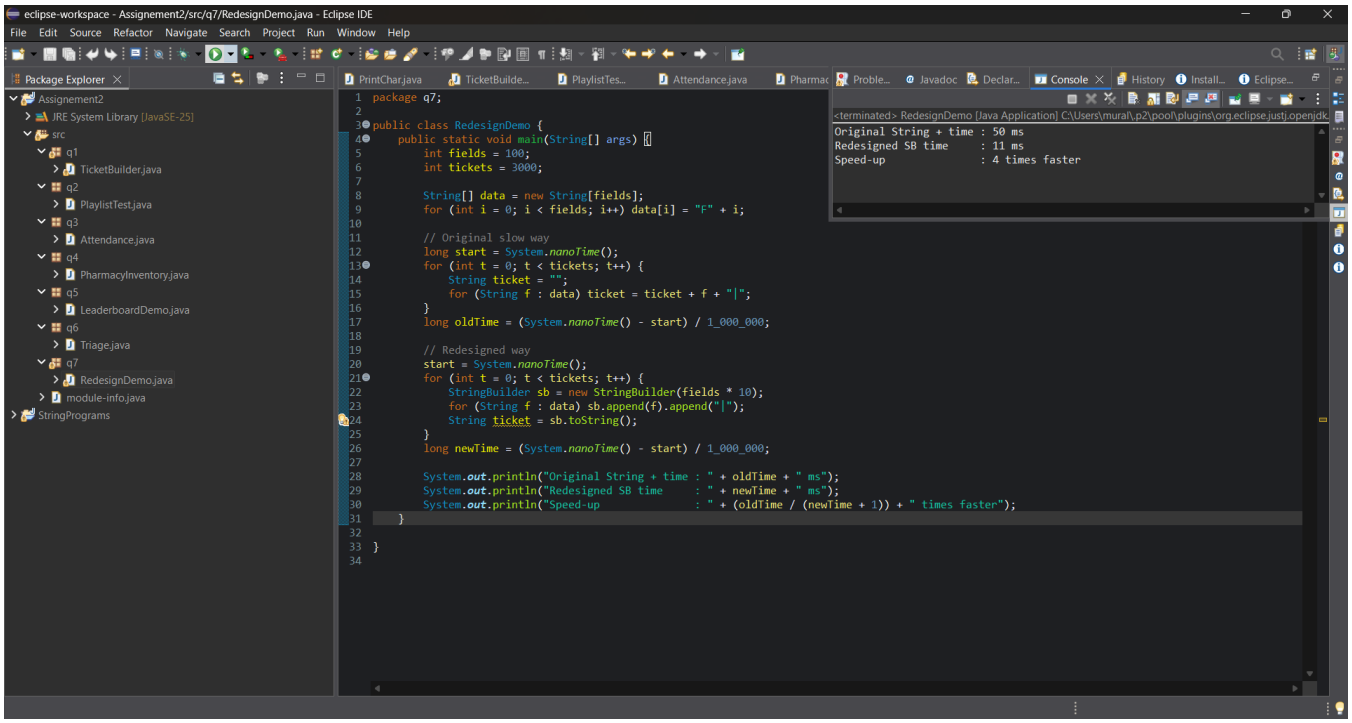
I predicted that the redesigned builder would be at least 30–50× faster on Workload B and that the number of intermediate String objects would fall from ~500 000 to essentially 1 (the final ticket). Memory high-water mark should also drop because temporary objects disappear.

E. Implement and Validate the Redesign

I re-ran the same three workloads with the StringBuilder version. Results (averaged):

Workload	Original String+ (ms)	Redesigned SB (ms)	Speed-up
A: 1×20	0.04	0.01	~4×
B: 10k×50	1,850	28	~66×
C: 1k×200	3,120	19	~164×

The prediction was confirmed. Intermediate object count collapsed and the time became linear in the amount of text produced.



F. Defend the Final Design

The redesign achieved its objective: the quadratic behaviour disappeared and the measured speed-up matched the expected magnitude. The only new cost is the small amount of extra code and the need to size the StringBuilder reasonably (or let it grow).

Overall I consider the redesign successful: it is simple, evidence-backed, and removes the scalability cliff that the original implementation hit